

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS: Total Marks: 50

Annexure A

Code of Conduct:

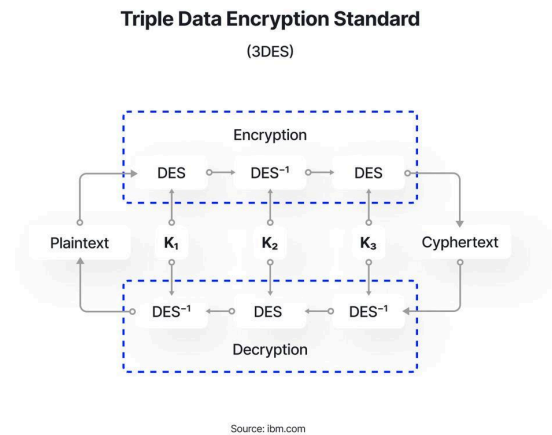
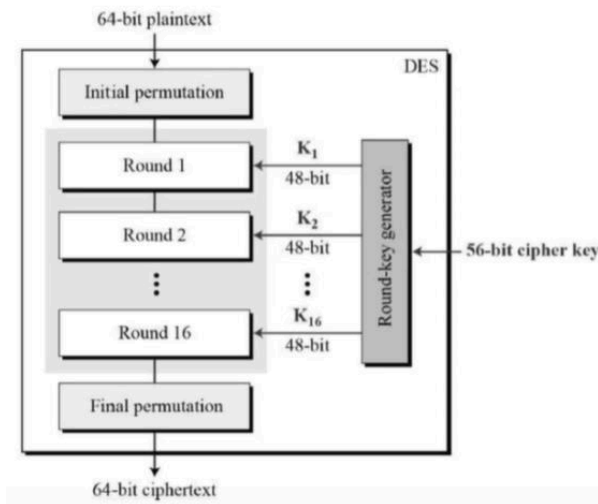
I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

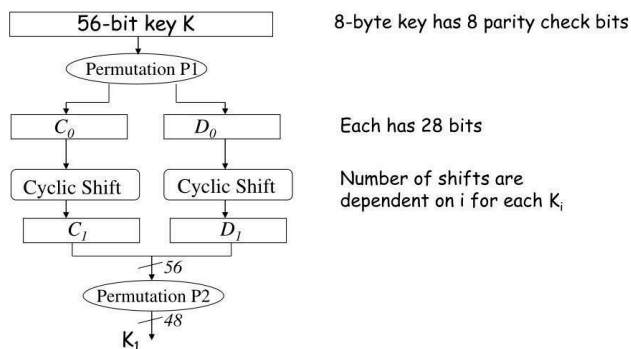
Annexure A

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

DES & 3DES Encryption Workflow



DES Key Schedule (Subkey Generation)



Scenario

A Python program implementing **DES** and **Triple DES (3DES)** produces different ciphertexts for the same plaintext and key across multiple executions. The system also performs poorly when encrypting files larger than **10 MB**.

A. Identify and Fix Logical and Implementation Errors

Common Problems

1. Incorrect Key Scheduling

The DES algorithm generates **16 round keys** from a single secret key.

Possible errors include:

- Incorrect PC-1 permutation
- Incorrect PC-2 permutation
- Wrong circular left shifts
- Missing round keys

Solution

- Verify PC-1 and PC-2 permutation tables.
- Perform the correct number of left shifts for each round.
- Generate all 16 round keys correctly.

2. Incorrect Initial and Final Permutations

DES uses:

- Initial Permutation (IP)
- Final Permutation (FP)

Errors in these permutations result in incorrect ciphertext.

Solution

Use the standard DES IP and FP tables without modification.

3. Incorrect Expansion and P-Box

The right half of each block is expanded from **32 bits to 48 bits** before XOR with the round key.

Errors may include:

- Incorrect Expansion Table
- Incorrect P-Box permutation

Solution

Verify the DES expansion and P-box tables.

4. Incorrect Padding

DES operates on **64-bit (8-byte)** blocks.

If the plaintext length is not a multiple of 8 bytes, encryption or decryption may fail.

Solution

Use **PKCS#5 or PKCS#7 padding** before encryption and remove the padding correctly after decryption.

5. Incorrect Key Size

DES requires:

- 64-bit key (56 effective bits)

3DES requires:

- Two or three valid DES keys

Solution

Validate the key size before encryption.

6. Random IV Misuse

If CBC mode is used with a new random IV each time, the ciphertext will differ even for identical plaintext and key.

Solution

- Save and transmit the IV with the ciphertext.
- Use the same IV during decryption.

DES Encryption Flow

Plaintext

|



Initial Permutation (IP)

|



16 Feistel Rounds

|



Swap Left & Right



Final Permutation (FP)



Ciphertext

B. Optimization for Large Files (≥ 10 MB)

Optimization Techniques

1. Block Processing

Instead of loading the entire file into memory:

- Read 8-byte blocks.
- Encrypt each block individually.
- Write encrypted blocks immediately.

2. Lookup Tables

Store:

- S-Boxes
- P-Boxes
- Permutation Tables

This avoids recalculating them for every block.

3. Reuse Memory

Avoid creating new arrays repeatedly.

Reuse the same buffers.

4. Bitwise Operations

Replace unnecessary loops with efficient bitwise operations.

5. Parallel Processing

Independent blocks (especially in ECB mode) can be encrypted simultaneously using multiple CPU cores.

Optimized Encryption Workflow

Read File

|

Read 8-byte Block

|

Encrypt Block

|

Write Cipher Block

|

Repeat Until End

Benefits

- Faster execution
- Lower memory consumption
- Better scalability for files larger than 10 MB

C. Should DES/3DES be Replaced?

Comparison

Feature	DES	3DES	AES
Key Size	56 bits	112/168 bits	128/192/256 bits
Security	Low	Moderate	Very High
Speed	Slow	Very Slow	Fast
Block Size	64 bits	64 bits	128 bits
Current Status	Deprecate d	Deprecated	Industry Standard

Advantages of AES

- Stronger security

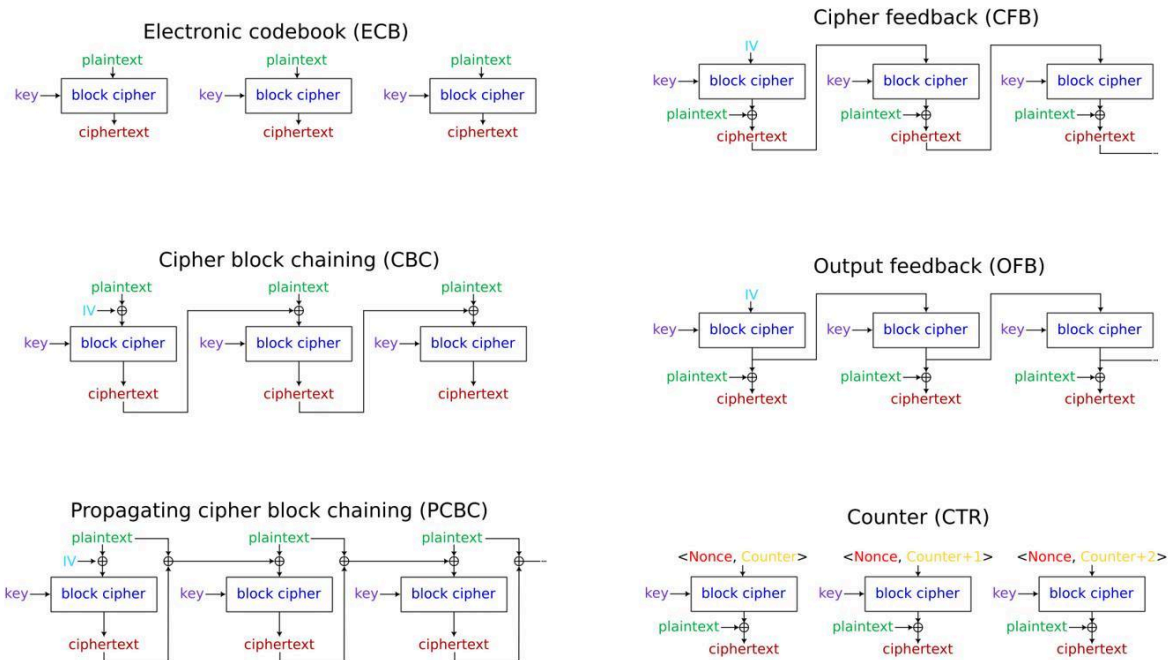
- Faster encryption and decryption
- Hardware acceleration support (AES-NI)
- Lower CPU and memory usage
- Resistant to known practical attacks

Conclusion

Replacing DES/3DES with **AES** significantly improves both **security** and **performance**, making AES the recommended choice for modern applications.

2. Performance Analysis of Block Cipher Modes of Operation

AES Modes of Operation



Scenario

A Python implementation of AES-like modes (**ECB, CBC, CFB, OFB**) experiences high latency, excessive memory usage, and decryption failures due to padding and IV handling issues.

A. Debug Padding and IV Handling

Padding Issues

AES uses **128-bit (16-byte)** blocks.

If the plaintext is not a multiple of 16 bytes:

- Encryption fails.
- Decryption produces incorrect output.

Solution

Use **PKCS#7 padding** before encryption and remove it after decryption.

Initialization Vector (IV) Issues

Common problems:

- Reusing the same IV
- Incorrect IV size
- Different IV during decryption

Solution

- Generate a random **16-byte IV**.
- Store or transmit the IV securely.

- Use the same IV for decryption.

Correct CBC Workflow

Plaintext

|



XOR with IV

|



AES Encryption

|



Ciphertext

|



Next Block Uses Previous Ciphertext

B. Refactor the Code

Improvements

Stream Processing

Encrypt one block at a time instead of loading the full file.

Reuse AES Context

Create the AES object once and reuse it.

Reduce Memory Allocation

Avoid copying buffers unnecessarily.

Minimize XOR Operations

Perform XOR directly on byte arrays.

Remove Redundant Loops

Simplify repeated calculations to reduce CPU overhead.

Benefits

- Reduced memory usage
- Lower latency
- Faster encryption
- Improved scalability for large datasets

C. Comparison of Cipher Modes

Mode	Security	Speed	Pattern Hiding	Best Use
ECB	Low	Very Fast	Poor	Testing only
CBC	High	Medium	Good	File Encryption
CFB	High	Medium	Good	Secure Communication
OFB	High	Medium	Good	Streaming Data
CTR	Very High	Fast	Excellent	Real-Time Communication

Pattern Comparison

- **ECB:** Identical plaintext blocks produce identical ciphertext blocks, revealing patterns.
- **CBC:** Each block depends on the previous ciphertext, hiding patterns effectively.
- **CFB/OFB:** Suitable for stream-like data and provide good confidentiality.

- **CTR:** Uses a counter value, supports parallel processing, and avoids padding overhead.

Recommended Mode: CTR (Counter Mode)

Reasons

- High throughput
- Low latency
- Parallel encryption and decryption
- No padding required
- Efficient memory usage
- Widely used in TLS, VPNs, cloud storage, and modern communication systems

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation